ONLINE QUIZ SYSTEM

Software Architecture and Design Specification

Course: Software Engineering
Class: 5th Semester – Section H
Team: 13
Document ID: OQS-AD-03
Version: 1.0
Date: 28th Sept 2026

Team Members

Name	SRN
Shwetha S	PES1UG24CS452
Srishti Rao	PES1UG24CS470
Pratistha Majumder	PES1UG25CS831
Sharvari Naik	PES1UG24CS431

Document Control

Version	Date	Description
1.0	02-11-2026	Initial Architecture and Design Specification

1. Introduction

1.1 Purpose

This document describes the proposed software architecture and detailed design of the Online Quiz System. It explains the system components, interactions, database structure, API design, security mechanisms and sequence of operations.

It provides a technical reference for implementing the requirements defined in OQS-SRS-01.

1.2 Scope

The design covers:

- Participant quiz operations.
- Administrator authentication.
- Question management.
- Automatic score calculation.

- Result display.
- Data storage and retrieval.
- Security and error handling.

1.3 Technology Stack

Layer	Technology
Frontend	HTML, CSS, JavaScript
Backend	Node.js
Backend Framework	Express.js
Database	MySQL
API Communication	REST APIs using JSON
Version Control	Git and GitHub

2. System Architecture

2.1 Architecture Pattern

The Online Quiz System follows a **three-tier architecture**.

The three tiers are:

1. Presentation Layer

This layer provides the user interface using HTML, CSS and JavaScript. Participants and administrators interact with the system through a web browser.

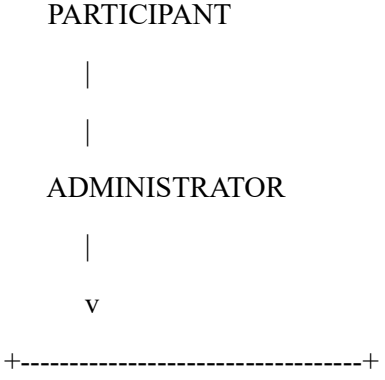
2. Application Layer

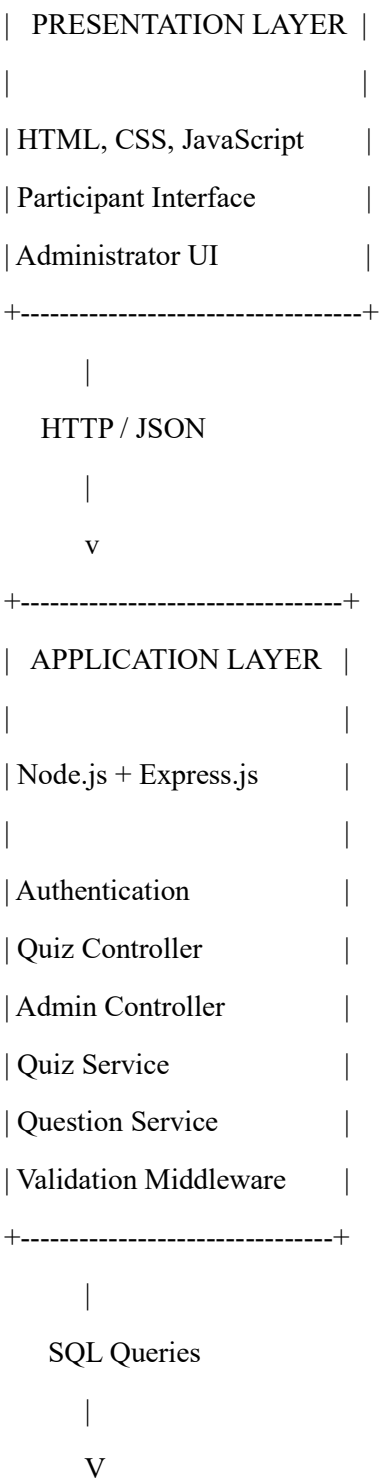
This layer is implemented using Node.js and Express.js. It handles requests, performs validation, executes business logic, calculates scores and controls access to administrative operations.

3. Database Layer

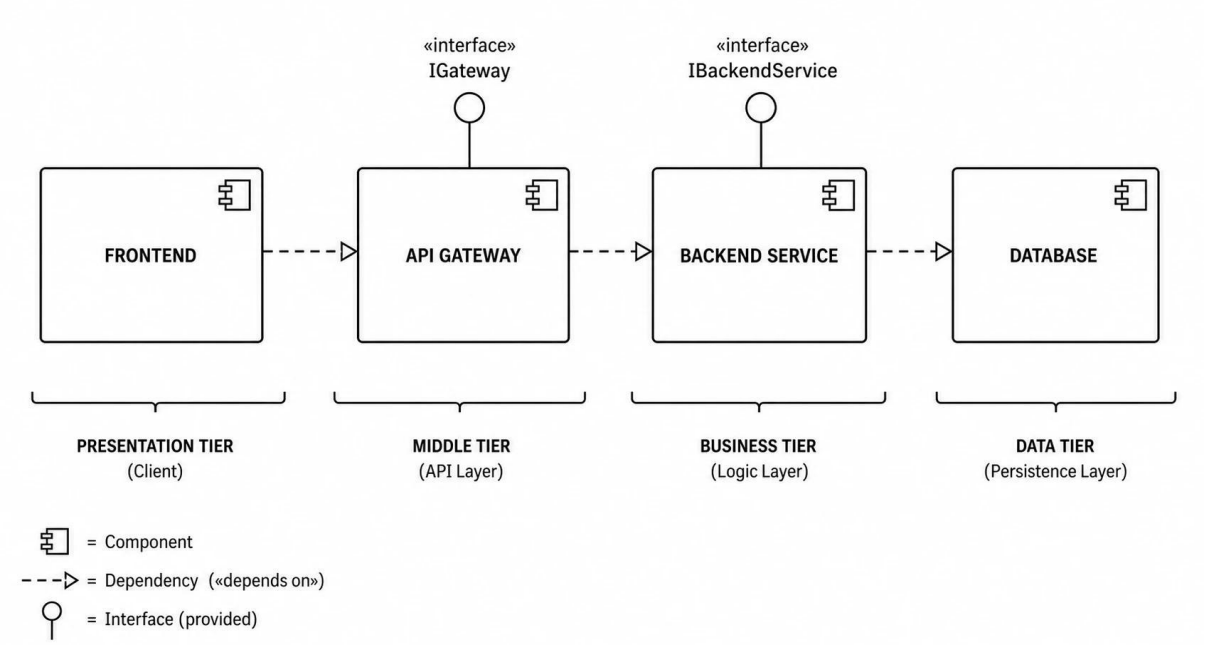
This layer uses MySQL to store administrator details, quizzes, questions, options, attempts and participant responses.

2.2 Architecture Diagram




PRESENTATION LAYER
HTML, CSS, JavaScript
Participant Interface
Administrator UI
HTTP / JSON
APPLICATION LAYER
Node.js + Express.js
Authentication
Quiz Controller
Admin Controller
Quiz Service
Question Service
Validation Middleware
SQL Queries

```
+------------------------------+
|    DATABASE LAYER            |
|                              |
|      MySQL                   |
|                              |
| Administrators               |
| Quizzes                      |
| Questions                    |
| Options                      |
| Attempts                     |
| Responses                    |
+------------------------------+
```



2.3 Component Descriptions

Component	Responsibility
Participant Interface	Displays quiz information, questions, options and results.
Administrator Interface	Provides login and question-management functionality.
Express API	Receives frontend requests and sends responses.
Authentication Middleware	Verifies administrator identity and access permissions.
Quiz Controller	Handles quiz-related HTTP requests.

Admin Controller	Handles administrator operations.
Quiz Service	Implements quiz submission and score calculation.
Question Service	Implements question creation, viewing, editing and deletion.
Validation Middleware	Checks request data before processing.
MySQL Database	Stores and retrieves system information.

2.4 Requirement Traceability

Requirement	**Architecture Component**
FR-01, FR-02	Participant Interface and Quiz Controller
FR-03, FR-04	Participant Interface and Quiz Service
FR-05, FR-06	Quiz Controller and Quiz Service
FR-07	Result Interface and Quiz Service
FR-08	Authentication Middleware
FR-09 to FR-13	Admin Controller, Question Service and Validation Middleware
NFR-01 to NFR-10	Overall architecture, database design and validation mechanisms
SEC-01 to SEC-06	Authentication, authorisation, input validation and secure database access

3. Detailed Design

3.1 Frontend Design

The frontend is developed using HTML, CSS and JavaScript.

The main pages are:

Participant Pages

1. Quiz Start Page
2. Participant Details Page

3. Quiz Attempt Page
4. Quiz Result Page

Administrator Pages

1. Administrator Login Page
2. Question Management Page
3. Add Question Page
4. Edit Question Page

JavaScript is responsible for handling user interactions, navigation and communication with backend APIs.

Client-side validation may be used to improve usability. However, all important validation must also be performed on the server.

3.2 Backend Design

The backend is developed using Node.js and Express.js.

The backend consists of the following modules:

Route Module: Defines API endpoints and connects requests to the appropriate controllers.

Controller Module: Handles incoming requests and prepares HTTP responses.

Service Module: Implements application logic such as question management and score calculation.

Authentication Middleware: Protects administrative routes.

Validation Middleware: Validates request data before processing.

Database Module: Establishes the connection to MySQL and executes database operations.

3.3 Suggested Project Structure

ONLINE-QUIZ-SYSTEM/

```
│
├── frontend/
│   ├── index.html
│   ├── quiz.html
│   ├── result.html
│   ├── admin-login.html
│   ├── admin-dashboard.html
│   │
│   ├── css/
```

```
│   │   └── styles.css
│   │
│   └── js/
│       ├── api.js
│       ├── quiz.js
│       └── admin.js
│
├── backend/
│   ├── app.js
│   ├── routes/
│   ├── controllers/
│   ├── services/
│   ├── middleware/
│   ├── repositories/
│   └── config/
│
├── database/
│   ├── schema.sql
│   └── seed.sql
│
├── tests/
│
└── README.md
```

This is the proposed structure. The team may adjust filenames and folders during implementation while maintaining the same separation of responsibilities.

4. Database Design

4.1 Database Overview

MySQL is used as the relational database management system.

The database stores administrator information, quiz details, questions, answer options, participant attempts and submitted responses.

4.2 Database Tables

1. Administrators

Attribute	Description
admin_id	Primary key
username	Administrator username
password_hash	Securely hashed password
role	Administrator role

2. Quizzes

Attribute	Description
quiz_id	Primary key
title	Quiz title
description	Quiz description
status	Quiz availability status

3. Questions

Attribute	Description
question_id	Primary key
quiz_id	Foreign key
question_text	Question content
marks	Marks assigned to the question

4. Options

Attribute	Description
option_id	Primary key
question_id	Foreign key
option_text	Answer option
is_correct	Indicates the correct option

5. Attempts

Attribute	Description
attempt_id	Primary key

quiz_id	Foreign key
participant details	Required participant information
started_at	Attempt start time
submitted_at	Submission time
score	Calculated score

6. Responses

Attribute	**Description**
response_id	Primary key
attempt_id	Foreign key
question_id	Foreign key
selected_option_id	Selected answer option

4.3 Database Relationships

- One quiz can contain multiple questions.
- One question can have multiple answer options.
- One quiz can have multiple participant attempts.
- One attempt can contain multiple responses.
- Each response refers to a question and the selected option.
- Each question belongs to a quiz.

Primary keys and foreign keys shall be used to maintain referential integrity.

4.4 Data Integrity

1. Each table shall have a primary key.
2. Foreign keys shall maintain relationships between related tables.
3. Required fields shall not accept null values.
4. Parameterised queries shall be used for database operations.
5. Quiz scores shall be calculated on the backend.
6. Database transactions shall be used where multiple related records must be saved together.

5. API Design

5.1 API Overview

The frontend communicates with the backend using REST-style APIs.

The base API path is:

/api/v1

Requests and responses shall use JSON.

5.2 Participant APIs

HTTP Method	Endpoint	Purpose
GET	/api/v1/quizzes/	Retrieve quiz information
POST	/api/v1/attempts	Create a quiz attempt
GET	/api/v1/attempts//questions	Retrieve quiz questions
PUT	/api/v1/attempts//responses/	Save a selected answer
POST	/api/v1/attempts//submit	Submit quiz and calculate score
GET	/api/v1/attempts//result	Retrieve quiz result

5.3 Administrator APIs

HTTP Method	Endpoint	Purpose
POST	/api/v1/admin/login	Administrator login
GET	/api/v1/admin/questions	View questions
POST	/api/v1/admin/questions	Add a question
PUT	/api/v1/admin/questions/	Edit a question
DELETE	/api/v1/admin/questions/	Delete a question

All administrator question-management APIs must require authentication and authorisation.

5.4 HTTP Status Codes

Status Code	Meaning
200	Request successful
201	Resource created
400	Invalid request
401	Authentication required or invalid
403	Access forbidden
404	Resource not found

409	Conflict
500	Internal server error

5.5 Example JSON Response

Successful response:

```
{
  "message": "Question added successfully",
  "data": {
    "questionId": 101
  }
}
```

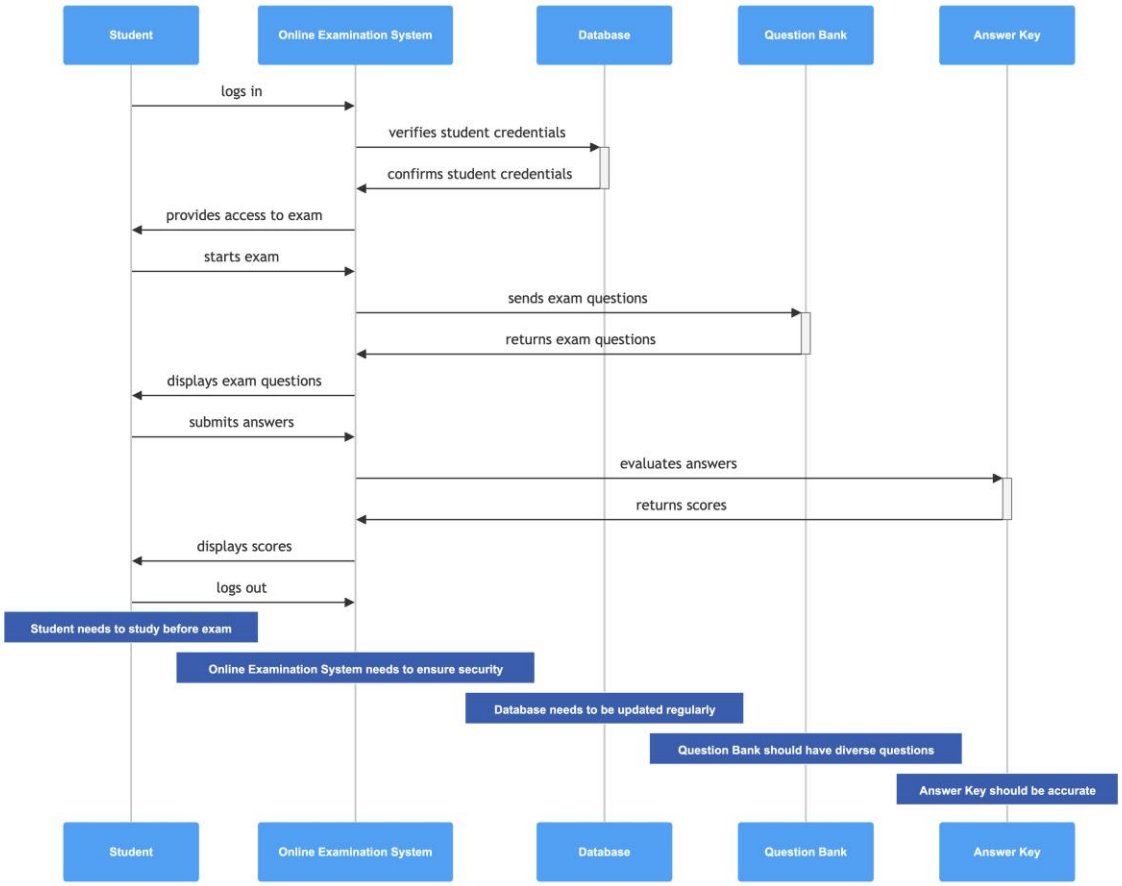
Validation error:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please check the submitted fields"
  }
}
```

6. Sequence Diagrams

6.1 Participant Quiz Submission Sequence Diagram

Description: This diagram represents the interaction between a participant, frontend, backend and database when a quiz is submitted.



The sequence of operations is:

1. Participant selects answers.
2. Browser sends selected answers to the backend.
3. Backend validates the submitted data.
4. Quiz Service saves the responses.
5. Participant selects Submit Quiz.
6. Browser sends a submission request.
7. Backend retrieves submitted responses and correct answers.
8. Quiz Service calculates the score.
9. Backend stores the final score and submission status.
10. Backend sends the result to the browser.
11. Browser displays the result to the participant.

PlantUML source:

@startuml

actor Participant

participant "Browser Interface" as UI

participant "Express API" as API

participant "Quiz Service" as QS

database "MySQL Database" as DB

Participant -> UI: Select answers

UI -> API: Save responses

API -> QS: Validate and save

QS -> DB: Store responses

DB --> QS: Confirmation

QS --> API: Success

API --> UI: Response saved

Participant -> UI: Submit quiz

UI -> API: Submit attempt

API -> QS: Evaluate attempt

QS -> DB: Retrieve answers and answer key

DB --> QS: Answer data

QS -> QS: Calculate score

QS -> DB: Save score and status

DB --> QS: Confirmation

QS --> API: Result summary

API --> UI: Return result

UI --> Participant: Display score

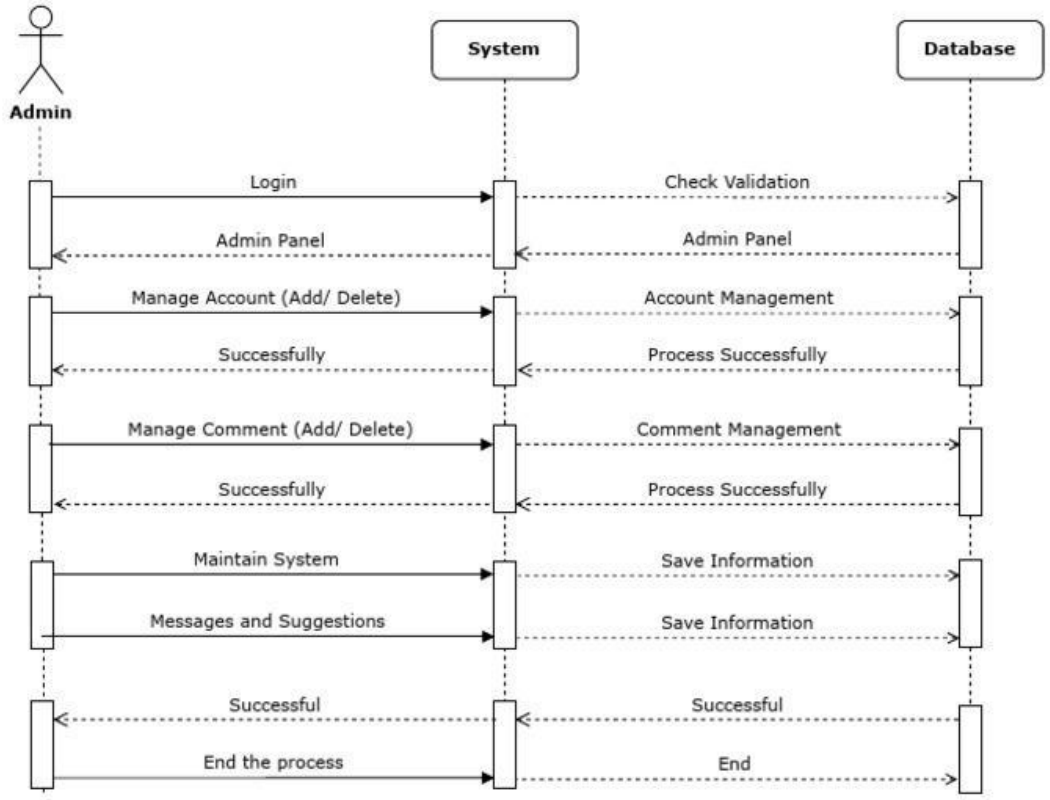
@enduml

6.2 Administrator Question Management Sequence Diagram

Description: This diagram represents administrator authentication and the addition of a new question. The sequence of operations is:

1. Administrator opens the login page.
2. Administrator enters username and password.
3. Backend verifies the credentials.
4. Backend grants access if credentials are valid.
5. Administrator opens question management.
6. Administrator enters question details and options.

7. Frontend sends the question data to the backend.
8. Authentication middleware verifies the administrator session.
9. Backend validates the question details.
10. Question Service saves the question in MySQL.
11. Backend returns a success response.
12. Frontend displays a confirmation message.



PlantUML source:

@startuml

actor Administrator

participant "Admin Interface" as UI

participant "Express API" as API

participant "Authentication Middleware" as AUTH

participant "Question Service" as QS

database "MySQL Database" as DB

Administrator -> UI: Enter login credentials

UI -> API: Send login request

API -> AUTH: Verify credentials

AUTH -> DB: Retrieve administrator

DB --> AUTH: Administrator record

AUTH -> AUTH: Verify password hash

AUTH --> API: Authentication result

API --> UI: Login response

Administrator -> UI: Enter question details

UI -> API: Send add-question request

API -> AUTH: Verify administrator access

AUTH --> API: Authorised

API -> QS: Validate and create question

QS -> DB: Insert question and options

DB --> QS: Confirmation

QS --> API: Question created

API --> UI: Success response

UI --> Administrator: Display confirmation

@enduml

7. Security Architecture

7.1 Authentication

Administrator authentication shall be performed by the backend.

The system shall verify administrator credentials before allowing access to restricted operations.

Passwords shall be stored as salted password hashes using a suitable password-hashing library. Plaintext passwords shall not be stored.

7.2 Authorisation

Only authenticated administrators shall be allowed to add, view, edit or delete questions through administrative APIs.

Participant access shall not grant administrator privileges.

7.3 Input Validation

All client-supplied data shall be validated on the backend.

Parameterised SQL queries shall be used to reduce SQL injection risks.

7.4 Data Protection

- HTTPS shall be used in deployed environments.
- Database credentials shall be stored outside source code.
- Sensitive information shall not be included in client-facing error messages.
- Correct answers shall not be exposed before quiz submission.
- Database access shall follow the principle of least privilege.

7.5 Security Requirement Mapping

Security Requirement	Design Mechanism
SEC-01	Authentication middleware
SEC-02	Password hashing
SEC-03	Input validation and parameterised SQL
SEC-04	HTTPS and secure session configuration
SEC-05	Centralised error handling
SEC-06	Role-based authorisation

8. Error Handling

8.1 Error Handling Strategy

The backend shall validate requests and return appropriate HTTP status codes.

Error Condition	Expected Behaviour
Missing required fields	Return validation error
Invalid login credentials	Reject login
Unauthorised admin request	Return 401 or 403
Question not found	Return 404
Database unavailable	Return a safe server error

Unexpected application error	Return a generic error message

8.2 Error Response Format

```
{
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "The requested resource was not found"
  }
}
```

The application shall not expose database credentials, SQL queries or server stack traces to users.

9. Deployment Architecture

The browser loads the frontend application. The frontend communicates with the Express.js server through APIs. The backend connects to the MySQL database.

The MySQL database shall not be directly accessible from the participant's browser.

The backend shall use environment variables for database credentials and other sensitive configuration.

10. Design Constraints and Assumptions

1. The first version supports MCQ-based quizzes.
2. The first version uses one correct answer per question unless the team agrees to extend the scoring model.
3. Administrator accounts will be configured for the application.
4. The system requires a working database connection.
5. Quiz duration, retake policy, negative marking and question randomisation must be confirmed before implementation.
6. The team shall follow the same requirement identifiers in the SRS, Test Plan and implementation.

11. Requirement-to-Design Traceability

Requirement	**Design Element**
FR-01, FR-02	Participant start page and attempt creation API
FR-03, FR-04	Question display and response-save API
FR-05, FR-06	Submission endpoint and Quiz Service
FR-07	Result endpoint and result interface
FR-08	Administrator login and authentication middleware

FR-09, FR-10, FR-11, FR-12	Question-management APIs and Question Service
FR-13	Backend validation middleware
NFR-01, NFR-02	Application performance and load testing
NFR-03, NFR-04	Password hashing and access-control mechanisms
NFR-05, NFR-09	Response persistence and database integrity
NFR-06, NFR-07	Frontend usability and browser compatibility
NFR-08	Modular application structure
NFR-10	Centralised error handling
SEC-01 to SEC-06	Security architecture and middleware

12. Conclusion

This Architecture and Design Specification defines the proposed technical structure of the Online Quiz System using HTML, CSS, JavaScript, Node.js, Express.js and MySQL.

It provides a foundation for implementation by describing the architecture, components, database, APIs, sequence flows, security mechanisms and error-handling approach.

The team shall review and approve the design before implementation.